

## Quality Lab Manual



# Operating System Lab 2ICPC204

### Experiment No. 6

## Program based on Preemptive CPU Scheduling Algorithms.

**Course Instructor -**

**PROF.MADHURI J SIDDHARAPU**  
**ASSISTANT PROFESSOR**

## **Experiment No. 6**

**Expt No : 6 (a)**

**Title** : Program based on Preemptive CPU Scheduling Algorithms.

**Aim** : Write a C program to implement Priority scheduling algorithm.

**Description** :

**Priority Scheduling** is a method of scheduling processes that is based on priority. In this algorithm, the scheduler selects the tasks to work as per the priority. Process with the highest priority is to be executed first and so on.

Processes with the same priority are executed on first come first served basis. Priority can be decided based on memory requirements, time requirements or any other resource requirement.

Advantages of Priority Scheduling Algorithm:

1. This provides a good mechanism where the relative importance of each process may be precisely defined.

Disadvantages of Priority Scheduling Algorithm:

1. If high-priority processes use up a lot of CPU time, lower-priority processes may starve and be postponed indefinitely. The situation where a process never gets scheduled to run is called starvation.
2. Another problem is deciding which process gets which priority level assigned to it.

**Algorithm:**

**Step 1:** Start the Program.

**Step 2:** Input the number of processes.

**Step 3:** Input the burst time and priority for each process.

**Step 4:** Sort the element on the basis of priority.

**Step 5:** Print order of execution of their process with their time stamp (wait time and turnaround time).

**Step 6:** End the Program.

**Program:**

```
#include<stdio.h>

#include<conio.h>

#define max 30
```

```
void main()
{
    int bt[max],p[max],wt[max],tat[max],pr[max];
    int i,j,n,total=0,pos,temp,avg_wt,avg_tat;
    clrscr();
    printf("Enter Total Number of Process:");
    scanf("%d",&n);
    printf("\nEnter the Burst Time and Priority of process\n");
    for(i=0;i<n;i++)
    {
        printf("\nP[%d]\n",i+1);
        printf("Burst Time:");
        scanf("%d",&bt[i]);
        printf("Priority:");
        scanf("%d",&pr[i]);
        p[i]=i+1;      //contains process number
    }

    //sorting burst time, priority and process number in ascending order using selection
    sort
    for(i=0;i<n;i++)
    {
        pos=i;
        for(j=i+1;j<n;j++)
        {
            if(pr[j]<pr[pos])
                pos=j;
        }
    }
```

---

```
temp=pr[i];
pr[i]=pr[pos];
pr[pos]=temp;

temp=bt[i];
bt[i]=bt[pos];
bt[pos]=temp;

temp=p[i];
p[i]=p[pos];
p[pos]=temp;
}

wt[0]=0; //waiting time for first process is zero

//calculate waiting time
for(i=1;i<n;i++)
{
    wt[i]=0;
    for(j=0;j<i;j++)
        wt[i]= wt[i]+bt[j];

    total= total+wt[i];
}

avg_wt=total/n;    //average waiting time
total=0;
```

```
printf("\nPriority\t Process\t Burst Time \tWaiting Time\tTurnaround Time");  
for(i=0;i<n;i++)  
{  
    tat[i]=bt[i]+wt[i];    //calculate turnaround time  
    total=total+tat[i];  
    printf("\n%d\t\t P[%d]\t\t %d\t\t %d\t\t%d",pr[i],p[i],bt[i],wt[i],tat[i]);  
}  
avg_tat=total/n;    //average turnaround time  
printf("\n\nAverage Waiting Time=%d\n",avg_wt);  
printf("\n\nAverage Turnaround Time=%d\n",avg_tat);  
getch()  
}
```

**Output:**

Enter total number of process: 4

Enter the burst time and priority of process

P[1]

Burst time: 6

Priority: 3

P[2]

Burst time: 8

Priority: 1

P[3]

Burst time: 10

Priority: 4

P[4]

Burst time: 7

Priority: 2

---

| Priority | Process | Burst Time | Waiting Time | Turn Around Time |
|----------|---------|------------|--------------|------------------|
| 1        | P[2]    | 8          | 0            | 8                |
| 2        | P[4]    | 7          | 8            | 15               |
| 3        | P[1]    | 6          | 15           | 21               |
| 4        | P[3]    | 10         | 21           | 31               |

Average waiting time is = 11

Average Turn Around Time is = 18

**Conclusion :**

Thus the C program to implement CPU scheduling algorithm for priority scheduling was executed successfully.

**Expt No: 6 (b)**

**Title** : Program based on Preemptive CPU Scheduling Algorithms.

**Aim** : Write a C program to implement Shortest Remaining Time First algorithm.

**Description** :

The Preemptive version of Shortest Job First(SJF) scheduling is known as **Shortest Remaining Time First (SRTF)**. With the help of the SRTF algorithm, the process having the smallest amount of time remaining until completion is selected first to execute.

So basically in SRTF, the processes are scheduled according to the shortest remaining time.

In the **SRTF scheduling algorithm**, the execution of any process can be stopped after a certain amount of time. On arrival of every process, the short-term scheduler schedules those processes from the list of available processes & running processes that have the least remaining burst time.

After all the processes are available in the ready queue, then, No preemption will be done and then the algorithm will work the same as SJF scheduling.

Advantages of Shortest Remaining Time First (SRTF):

1. SRTF is optimal and guarantees the minimum average waiting time.
2. It provides a standard for other algorithms since no other algorithm performs better than it.

Disadvantages of Shortest Remaining Time First (SRTF):

1. It cannot be implemented practically since burst time of the processes can not be known in advance.
2. It leads to starvation for processes with larger burst time.
3. Priorities cannot be set for the processes.
4. Processes with larger burst time have poor response time.

**Algorithm:**

**Step 1:** Start the Program.

**Step 2:** Input the number of processes.

**Step 3:** Input the burst time for each process.

**Step 4:** Sort the element on the basis of Burst time.

**Step 5:** Print order of execution of their process with their time stamp (wait time and turnaround time).

**Step 6:** End the Program.

---

### **Program**

```
#include<stdio.h>

#include<conio.h>

#define max 30

void main()

{

    int bt[max],p[max],wt[max],tat[max],i,j,n,total=0,pos,temp;

    float avg_wt,avg_tat;

clrscr();

    printf("Enter total number of process:");

    scanf("%d",&n);

    printf("\nEnter Burst Time:");

    for(i=0;i<n;i++)

    {

        printf("P[%d]:",i+1);

        scanf("%d",&bt[i]);

        p[i]=i+1;

    }

    //sorting of burst times

    for(i=0;i<n;i++)

    {

        pos=i;

        for(j=i+1;j<n;j++)

        {

            if(bt[j]<bt[pos])
```



```

    pos=j;
}

temp=bt[i];
bt[i]=bt[pos];
bt[pos]=temp;

temp=p[i];
p[i]=p[pos];
p[pos]=temp;
}

wt[0]=0;

for(i=1;i<n;i++)
{
    wt[i]=0;
    for(j=0;j<i;j++)
        wt[i]=wt[i]+bt[j];

    total=total+wt[i];
}
avg_wt=(float)total/n;
total=0;
printf("\nProcessID\t Burst Time\tWaiting Time\tTurnaround Time");
for(i=0;i<n;i++)
{
```

```
    tat[i]=bt[i]+wt[i];  
    total=total+tat[i];  
    printf("\nP[%d]\t\t%d\t\t%d\t\t%d",p[i],bt[i],wt[i],tat[i]);  
}  
avg_tat=(float)total/n;  
printf("\nAverage Waiting Time=%f\n",avg_wt);  
printf("\nAverage Turnaround Time=%f",avg_tat);  
getch();  
}
```

**Output**

Enter total number of process: 4

Enter Burst Time:

P[1] : 5

P[2] : 3

P[3] : 1

P[4] : 6

| Process ID | Burst Time | Waiting Time | Turn Around Time |
|------------|------------|--------------|------------------|
| P[3]       | 1          | 0            | 1                |
| P[2]       | 3          | 1            | 4                |
| P[1]       | 5          | 4            | 9                |
| P[4]       | 6          | 9            | 15               |

Average waiting time is = 3.50000

Average Turn Around Time is = 7.250000

**Conclusion :**

Thus the C program to implement CPU scheduling algorithm for Shortest Remaining Time First was executed successfully.

**Procedure:**

1. Install Microsoft Windows and the Turbo C compiler on your computer.
2. Open Turbo C and create a new file for your program.
3. Ask the user to enter the number of processes they want to schedule.
4. For each process, ask for the, burst time, and priority.
5. Choose which scheduling algorithm to use.
6. Execute the chosen algorithm and display the results.
7. Compare these results to see how each algorithm performs.
8. Discuss the pros and cons of each algorithm based on the results you observed.

**Practice Problems Based on Preemptive CPU Scheduling Algorithms**

Note: In Priority Scheduling, the process with the lowest priority number gets executed first (highest priority), whereas the process with the highest priority number gets executed later (lowest priority).

**Task 1: Priority Scheduling Algorithm**

Consider the following set of processes. Execute the Priority Scheduling program and find the Average Waiting Time and Average Turnaround Time.

| Process ID | Arrival Time | Burst Time | Priority |
|------------|--------------|------------|----------|
| P1         | 0            | 5          | 3        |
| P2         | 1            | 8          | 1        |
| P3         | 2            | 6          | 4        |
| P4         | 4            | 2          | 2        |

**Task 2: Priority Scheduling Algorithm**

Consider the following set of processes. Execute the Priority Scheduling program and find the Average Waiting Time and Average Turnaround Time.

| Process ID | Arrival Time | Burst Time | Priority |
|------------|--------------|------------|----------|
| P1         | 0            | 7          | 4        |
| P2         | 2            | 3          | 1        |
| P3         | 4            | 9          | 5        |
| P4         | 6            | 4          | 2        |
| P5         | 7            | 6          | 3        |

**Task 3: SRTF Scheduling Algorithm**

Consider the following set of processes. Execute the SRTF scheduling program and find the Average Waiting Time and Average Turnaround Time.

| Process ID | Arrival Time | Burst Time |
|------------|--------------|------------|
| P1         | 0            | 7          |
| P2         | 2            | 4          |
| P3         | 4            | 1          |
| P4         | 5            | 4          |

**Task 2: SRTF Scheduling Algorithm**

Execute the SRTF scheduling program for the following processes.

| Process ID | Arrival Time | Burst Time |
|------------|--------------|------------|
| P1         | 0            | 8          |
| P2         | 1            | 3          |
| P3         | 3            | 6          |

Find:

Average Waiting Time

Average Turnaround Time

**References –****a. Textbook –**

- "Operating System Concepts", Abraham Silberschatz, Henry Korth, S. Sudarshan- 10th Edition (2018), Publisher: Wiley
- "Operating Systems: Principles and Practice"- Thomas Anderson and Michael Dahlin, 2nd Edition (2018), Publisher: Recursive Books

**b. Online references –**

- <https://www.geeksforgeeks.org/cpu-scheduling-set-1-introduction/>
- [https://www.tutorialspoint.com/operating\\_system/os\\_scheduling\\_algorithms.htm](https://www.tutorialspoint.com/operating_system/os_scheduling_algorithms.htm)
- <https://github.com/search?q=cpu+scheduling+notebook>

**Expected Oral Questions –**

1. What is Priority Scheduling algorithm? How does the scheduler decide which process will execute first?
2. In Priority Scheduling, why is a lower priority number considered as a higher priority?
3. What is starvation in Priority Scheduling? How can starvation be prevented?
4. What is the difference between SJF and SRTF scheduling algorithms?
5. How does SRTF handle a newly arrived process with a shorter burst time than the currently executing process?
6. What are the advantages and disadvantages of SRTF scheduling algorithm?
7. Why is SRTF considered a preemptive scheduling algorithm?
8. How are waiting time and turnaround time calculated in Priority Scheduling and SRTF algorithms?